ASL Bridge

Real-time ASL <-> English communication

"No special request. No interpreter. No aid. Just talk."

Project Overview

Hackathon Submission - ConHacks 2026

Team: Kai - Bob - Max - Dave

36-hour hackathon - April 2026

github.com/naik26m3/signly-conhacks-2026

Table of Contents

About this document

This overview describes ASL Bridge as built during ConHacks 2026 - the architecture, the engineering decisions, the trade-offs we accepted in 36 hours, and where the project goes next. Code references throughout point to actual files in the repository.

1. Executive Summary

ASL Bridge is a mobile application that lets a Deaf person and a hearing person hold a face-to-face conversation in real time without an interpreter. The Deaf user signs naturally; the app recognises the sign, speaks the English translation aloud, and stores it in the chat thread. The hearing user replies with their voice; the app transcribes it, converts the transcript to ASL gloss, and (optionally) renders an animated avatar performing the ASL response.

Everything is built around one simple bar: end-to-end latency under two seconds. The whole pipeline - video capture, MediaPipe hand tracking, Gemini vision inference, ElevenLabs text-to-speech, persistence to PostgreSQL, and audio playback on the phone - is engineered to fit inside that budget.

The system is shipped as a containerised stack (FastAPI API + ARQ worker + Postgres + Redis + SeaweedFS + Prometheus/Grafana/Loki) and an Expo React Native client. The app exposes three modes: sign-to-speech translation, hearing-to-deaf avatar animation, and AI voice design. It uses Gemini 2.5 Flash for sign recognition and voice persona generation, MediaPipe HandLandmarker (VIDEO mode) for 21-point hand tracking, MediaPipe FaceLandmarker for spatial anchor points, and ElevenLabs Scribe for speech-to-text, Multilingual v2 for text-to-speech, and Voice Design API for custom AI-generated voices. Animation of the hearing-to-deaf path is handled by an embedded sign.mt avatar viewer.

2. The Problem We Set Out to Solve

An estimated 70 million people worldwide use a sign language as their first language. Despite that, every day they bump into hearing people who cannot sign - at the doctor, in customer service, at school, on the bus. Today's options are professional human interpreters (excellent but scarce, expensive, and requiring scheduling), pen-and-paper, or hand-typed messages on a phone. None of them feel like a real conversation.

What is broken about the existing options

- Interpreters need to be booked - useless for spontaneous interactions.
- Typing on a phone breaks eye contact, slows the exchange, and erases the visual richness of sign language.
- Existing translation apps mostly do one direction (sign-to-text) and demand studio-clean conditions, perfectly framed hands, and a small fixed vocabulary.
- None of them close the loop on the hearing person's side - the Deaf user still has to read text, which is slower and less natural than receiving signs.

Our north star

Design principle
If the conversation feels noticeably slower than two people both speaking English, we have failed. Latency, eye contact, and visual feedback matter more than vocabulary breadth at this stage.

3. Our Solution: Three Conversational Modes

The app exposes three screens, each managing its own conversation history via a shared SessionContext. All three tabs share the same session ID so a voice persona designed on the Voice tab can be selected for read-aloud on either conversation tab.

Translate (default tab)

A chat-style interface with two large action buttons at the bottom: a camera FAB (brown) and a microphone FAB (dark brown, red while recording). Tapping the camera opens a full-screen recorder; tapping the mic starts a voice recording. As soon as the result returns, a chat bubble slides up with the gloss, the natural English translation, and a speaker icon to replay the audio.

- Sign bubbles render on the LEFT, brown outline, with the ASL gloss as the bold primary line and the natural English translation underneath.
- Speech bubbles render on the RIGHT, solid brown, with the transcript as primary and the back-translated ASL gloss as secondary.
- While a sign clip is being processed we show a 'pending bubble' with a rotating set of Giphy meme GIFs - SpongeBob waiting, Skeleton 'where you at?', Titanic '84 years' - so the wait feels human, not stuck.
- On a successful sign result the app auto-plays the ElevenLabs TTS audio so the hearing person hears the translation immediately, without anyone tapping anything.
- A voice-reasoning chip shows the selected voice's tone, pace, and rationale. A history drawer (slide-in from left) shows conversation history with timestamps and a voice picker shortcut.

Animation (second tab)

The Animation page is the hearing-to-deaf complement. The hearing person types a message or taps the mic, and an embedded WebView shows an avatar performing the ASL translation. We use sign.mt as the avatar engine, but inject a CSS overlay at page load to hide their language picker, input bar, and FAB controls so only the avatar viewer is visible - our native Expo input drives the URL state.

```
// AnimationPage.jsx (excerpt)
function buildSignMtUrl(text) {
  const params = new URLSearchParams({ spl: 'en', sil: 'ase', text });
  return `${SIGN_MT_BASE}?${params.toString()}`;
}
<WebView injectedJavaScriptBeforeContentLoaded={HIDE_CHROME_JS}
         injectedJavaScript={HIDE_CHROME_JS} ... />
```

Because Angular hydrates after first paint, the injected script also installs a MutationObserver that re-applies the hide-chrome CSS whenever the DOM changes - without it, the sign.mt UI flickers back in after a few hundred milliseconds.

Why a WebView?
Building a full SignLM avatar pipeline was out of scope for 36 hours. sign.mt has a public, well-tested avatar viewer that already supports ASL gloss output, so we treat it as Phase-1 plumbing and plan to swap in our own avatar in Phase 2.

Voice Design (third tab)

The Voice page is a conversation UI for creating custom AI voices. The user describes the voice they want in plain English ('a calm, slow, warm older woman with a slight Southern accent'). Gemini generates a structured voice persona; ElevenLabs Voice Design API synthesises a sample clip. The created voice appears as a VoiceBubble with tags, description, sample text, and a play button.

- UserBubble (right): the user's description prompt.
- PendingBubble (left): a multi-phase animation cycling through 'Gemini designing...' -> 'ElevenLabs generating...' while both API calls run.
- VoiceBubble (left): the finished voice card with a speaker tag list, description blurb, and playable audio sample.
- Created voices are stored in SessionContext and appear in the voice picker under 'My Voices' on any tab, so the same persona can read aloud sign translations or animation captions.

```
// Voice flow
POST /api/v1/voice/design  { description }  -> { voice_id, name, preview_url }
POST /api/v1/voice/speak   { voice_id, text } -> mp3 audio stream
```

4. End-to-End System Architecture

The system splits into three planes: the mobile client, a synchronous FastAPI surface for fast metadata operations, and an asynchronous ARQ worker pool for everything that involves an ML model or third-party API call. Splitting these cleanly is what makes the perceived latency low: the client gets a 202 Accepted in under 100ms while the heavy work runs in parallel and is delivered via a Redis-backed result key.

Path 1 - Deaf to Hearing

```
Front camera (Expo CameraView, 480p H.264, max 10s)
  -> POST /api/v1/sign/recognize  (multipart/form-data)
     . API writes file to shared tmp volume
     . API enqueues ARQ job with shared_tmp_path
     . API returns { video_id, status: 'processing' }
  -> Worker:
     1. ffmpeg -> 480p H.264 (smaller payload, faster Gemini upload)
     2. MediaPipe HandLandmarker (VIDEO) -> 21-pt hand sequence
     3. MediaPipe FaceLandmarker -> facial anchor points
     4. Gemini 2.5 Flash recognises the sign using video + landmarks
     5. ElevenLabs TTS synthesises the English audio
     6. Result + audio_url written to Redis (sign:{video_id})
     7. Conversation + Message rows persisted to Postgres
  -> Client polls /api/v1/sign/result/{id} -> auto-plays audio
```

Path 2 - Hearing to Deaf

```
Mic (Expo Audio.Recording, m4a)
  -> POST /api/v1/speech/transcribe (synchronous, no worker)
     . ElevenLabs Scribe transcribes audio -> English text
     . hearing_to_deaf message persisted to Postgres
     . Returns { transcript }
  -> POST /api/v1/speech/gloss { text }
     . Gemini Flash returns ASL gloss notation
  -> Animation tab: WebView opens sign.mt with text=<transcript>
     . Avatar performs the sign in-app
```

Why split sync vs async this way?

- Sign recognition is the slowest hop (1.5-2.0s) because it bundles ffmpeg, MediaPipe and a Gemini round-trip. Putting it on a worker lets the API return in ~50ms and unblocks the UI.
- Speech transcription is fast enough (300-700ms) that an extra polling round would actually hurt UX, so it stays inline.
- Both paths share a single Conversation row keyed by X-Session-ID, so the chat history endpoint returns a unified, ordered thread.

5. Technology Stack

Mobile client

Runtime	Expo SDK 54, React Native 0.81, React 19
Camera	expo-camera 17 (CameraView, video mode, front + back facing)
Audio	expo-av 16 (Audio.Recording HIGH_QUALITY preset, Audio.Sound playback)
Video preview	expo-video 3 (useVideoPlayer + VideoView)
Animation	react-native-reanimated 4, native Animated API for bubbles
Avatar	react-native-webview 13 hosting sign.mt with chrome stripped
System TTS	expo-speech (device OS voices, grouped by language in voice picker)
State	React Context (SessionContext) - per-tab history + created voices
Routing	Three-tab manual switch in app/index.tsx (Translate / Animation / Voice)

Backend

API	FastAPI 0.115 + Uvicorn 0.30 (Python 3.11)
Worker	ARQ 0.25 (Redis-backed async task queue, max_jobs=2, timeout=120s)
Database	PostgreSQL 16 + asyncpg 0.31 + SQLAlchemy 2.0 (async) + Alembic
Cache / queue	Redis 7
Object store	SeaweedFS (master + volume + filer)
Computer vision	MediaPipe 0.10.35 (HandLandmarker + FaceLandmarker)
Vision LLM	Gemini 2.5 Flash + 2.5 Flash-Lite fallback (google-genai 1.74)
STT / TTS	ElevenLabs Scribe v1 + Multilingual v2
Observability	Prometheus, Loki + Promtail, Grafana, Langfuse LLM tracing
Container	Docker Compose: api, worker, postgres, redis, 3x seaweedfs, monitoring

Why these choices?

- Expo over bare React Native: needed to be running on the judges' phones in five minutes, with no Xcode/Android Studio dance.
- FastAPI: type hints + Pydantic + automatic OpenAPI at /docs gave the frontend a typed contract before the API existed.
- ARQ over Celery: lighter, async-native, Redis-only - no broker, no thread-pool gymnastics, shares the API's async ecosystem.
- MediaPipe over a custom CNN: zero training time, 21-point output is exactly the abstraction Gemini understands as JSON.
- Gemini 2.5 Flash: the only frontier vision model with latency low enough on short clips to fit our 2s bar in one round-trip.

6. Backend Deep Dive (FastAPI + ARQ)

The backend is laid out as a conventional FastAPI app with explicit modules per concern. The module map follows the principle that each file owns exactly one external dependency or one type of artefact:

```
backend/
  main.py                  # app factory, lifespan, router wiring
  server.py                # uvicorn entrypoint
  worker.py                # ARQ worker (sign recognition + TTS + DB)
  config/
    settings.py            # Pydantic Settings (loads .env)
    database.py            # async SQLAlchemy engine + Session
    redis.py               # Redis pool
    gemini.py              # google-genai client factory
    elevenlabs.py          # ElevenLabs client factory
    langfuse.py            # Langfuse observability client
  db/models.py             # SQLAlchemy: Conversation + Message
  routers/
    health.py  sign.py  speech.py  uploads.py  conversations.py  voice.py
  schemas/                 # Pydantic request/response models (incl. voice.py)
  services/
    storage.py             # SeaweedFS save/validate (save_bytes)
    inference.py           # Gemini: recognize_sign, gloss <-> english, voice design
    speech.py              # ElevenLabs: STT + TTS + voice design
    conversation.py        # DB helpers (upsert + insert)
    collector.py           # JSONL data-collection logging
  middleware/request_logger.py
  migrations/              # Alembic
```

Lifespan: services live on app.state

FastAPI's lifespan context manager wires up the heavy clients exactly once at startup and tears them down on shutdown. This is critical for two reasons: (1) the MediaPipe HandLandmarker takes ~400ms to load and we cannot afford to do it per request; (2) Gemini and ElevenLabs clients hold pooled HTTP connections.

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    await Database.connect()
    await RedisClient.connect()
    HandTracker.load()
    FaceTracker.load()
    gemini = GeminiClient.connect()
    elevenlabs = ElevenLabsClient.connect()
    app.state.inference = InferenceService(gemini=gemini, langfuse=langfuse)
    app.state.speech = SpeechService(elevenlabs=elevenlabs)
    yield
    HandTracker.unload(); FaceTracker.unload()
    await Database.disconnect(); await RedisClient.disconnect()
```

Endpoints

GET /api/v1/health	Service status + API key availability
POST /api/v1/sign/recognize	Multipart video; returns 202 + video_id
POST /api/v1/sign/detect-hands	Pre-flight MediaPipe check before recognise
GET /api/v1/sign/result/{video_id}	Poll Redis for the worker's output
GET /api/v1/sign/audio/{video_id}	Proxy TTS audio out of SeaweedFS
POST /api/v1/sign/corrections	Append a human-corrected gloss
POST /api/v1/speech/transcribe	Audio -> ElevenLabs Scribe -> transcript
POST /api/v1/speech/gloss	JSON { text } -> Gemini -> ASL gloss
GET /api/v1/conversations/{session_id}/messages	Full ordered chat thread

GET
/api/v1/conversations/{session
_id}/title

POST /api/v1/voice/design	Text description -> Gemini persona -> ElevenLabs voice
POST /api/v1/voice/speak	Synthesise speech with a created custom voice

Worker pipeline (worker.py: process_sign_video)

The ARQ worker function is the most complex single piece of code in the project. Its shape is deliberate: every step has a fail-soft fallback so we never block the user's result on a non-critical hop (TTS, DB persistence, observability). The worker runs in its own container, on its own Redis queue, with max_jobs=2 to avoid GPU/CPU contention.

```
1. Receive (video_id, content_type, session_id, shared_tmp_path)
2. Fast path: if shared_tmp_path exists -> use it (skip SeaweedFS download)
   Fallback: GET <filer>/videos/{id}.mp4 into a NamedTemporaryFile
3. ffmpeg -> 480p H.264 mp4 (libx264 crf=28 ultrafast, +faststart, AAC 64k)
4. HandTracker.process_video(...) -> landmark_sequence, landmarks_found
   - face landmarks every 4th frame, hand landmarks every 2nd frame
   - per-frame wrist velocity (dx, dy) for motion disambiguation
5. inference.recognize_sign(video, landmarks) -> { gloss, english, confidence }
   - inline base64 if <19MB, Files API otherwise
   - thinking_budget=0, automatic_function_calling disabled
6. ElevenLabs.text_to_speech.convert(...) -> mp3 bytes
7. save_bytes(audio, video_id, folder='audio', ext='mp3') -> SeaweedFS
8. upsert_conversation(session_id) + insert_message(deaf_to_hearing)
9. collector.log_inference -> data/raw/inferences.jsonl (silent training set)
10. redis.setex('sign:{video_id}', 3600, json) -> client polls and gets result
```

Fast-path optimisation: shared tmp volume

The naive worker design uploads the video to SeaweedFS in the API, then has the worker download it back. That double-hop costs ~150-300ms on a 4-8MB clip. We bypassed it by writing to a shared docker volume (/app/tmp_videos) directly inside the API handler and passing the path through to the worker as the shared_tmp_path argument. SeaweedFS still gets the bytes - in a BackgroundTask that runs concurrently - so the artefact is durable for replay, but the worker doesn't wait for it.

Why is this safe?
Both api and worker mount the same backend/ tree, so /app/tmp_videos is a real shared filesystem. The worker unlinks the file at the end. In the worst case (worker crash before unlink), the file is reaped by docker volume rotation.

7. Computer Vision: Hand and Face Tracking

models/handTracking.py wraps two MediaPipe Tasks landmarkers as singleton classes - HandTracker and FaceTracker. Both load lazily at backend startup and unload on shutdown. We expose three operations: a quick image-mode hand detect for the pre-flight UX, a video-mode landmark sequence for the inference pipeline, and a face-tracker that emits exactly nine named anchor points relevant to ASL placement.

Quick pre-flight: HandTracker.quick_detect

Before sending the video to Gemini, the camera-recorder posts the just-recorded clip to /api/v1/sign/detect-hands. The endpoint samples 6 evenly-spaced frames in IMAGE mode and returns true as soon as ONE of them contains a hand. On a 10-second clip this takes ~600ms and means a re-take prompt costs a tiny fraction of an inference call.

```
// camera-recorder.tsx (excerpt)
const [handsCheck, setHandsCheck] = useState<HandsCheck>('idle');
useEffect(() => {
  if (!previewUri) return;
  setHandsCheck('checking');
  detectHands(previewUri).then(r =>
    setHandsCheck(r.data.hands_detected ? 'detected' : 'missing'));
}, [previewUri]);
```

Video-mode landmarks: HandTracker.process_video

The full inference pass runs in VIDEO mode (which preserves temporal smoothing across frames), num_hands=2, and emits a JSON-friendly sequence. Each entry includes:

- frame index
- right and left hand landmarks - 21 [x, y, z] tuples each, normalised 0-1, rounded to 4 decimal places to keep the JSON small
- face: nine named landmarks - forehead, chin, nose_tip, left_cheek, right_cheek, mouth_left, mouth_right, left_eyebrow, right_eyebrow - sampled every 4th frame and carried forward (faces barely move, so 7-8 fps saves 2-3s of MediaPipe time)
- right_vel and left_vel: wrist deltas since previous sample. These are critical for motion disambiguation - distinguishing 9 from 19, COME from GO, HELLO from MY by the magnitude of wrist movement

Why both face and hand?

Sign linguistics 101

ASL is not just hands. The same handshape at the forehead vs at the chin is a different sign. Velocity magnitude separates fingerspelling (static) from numbers with motion modifiers. By including face anchor points and wrist velocity in the Gemini prompt, we get a model that can reason geometrically about WHERE the hand is, not just what shape it makes.

8. AI Inference: Gemini Prompt Engineering

services/inference.py is where the 'think hard' happens. We use Gemini 2.5 Flash as the primary model with 2.5 Flash-Lite as a retry fallback. On a clean inline path the round-trip is consistently 800-1200ms; on the Files API path it adds 2-4s to wait for the file to become ACTIVE.

Closed-set classification with a vocabulary prior

Hackathon constraints forced a key decision: do we try to recognise the full ASL lexicon (impossible reliably in 36h) or a small demo-friendly vocabulary (reliable, demoable, useful for the judge interaction)? We chose the latter. The recognition prompt explicitly enumerates five demo phrases and instructs Gemini to pick exactly one - never UNKNOWN.

```
1. HI WHAT YOUR NAME       -> 'Hi, what is your name?'
2. MY NAME K-A-I.          -> 'My name is Kai.'
3. HOW YOU.                -> 'How are you?'
4. PHO                     -> 'Pho.'
5. NICE MEET YOU           -> 'Nice to meet you.'
```

Anti-bias rule

Early prompts had Gemini gravitate toward fingerspelled phrases whenever it saw any letter-like handshape. We patched this with an explicit anti-bias clause that requires THREE separate letter handshapes in quick succession before classifying as a fingerspelled phrase, plus disambiguation rules for the most-confused pairs (HI vs MY, HOW vs YOU, NICE vs MEET, NAME alone is not enough to imply phrase 2 because phrase 1 also contains NAME).

Inline vs Files API

Files API is only worth using for videos > 19MB - most sign clips after 480p transcode are 1-4MB, so we embed the bytes directly via types.Blob. This skips the 'wait for ACTIVE' polling loop and shaves 2-4 seconds off the worst case. Above 19MB we fall back to Files API and poll for ACTIVE up to 30 seconds.

Latency knobs

- thinking_budget=0 - disables Gemini's chain-of-thought entirely. Costs us a small amount of accuracy on borderline samples, but cuts ~600-1200ms per call.
- automatic_function_calling.disable=True - avoids an internal SDK loop that occasionally hangs the request for 60-90s on otherwise short queries.
- 3.0 fps video sampling (Gemini's default is 1 fps) - captures sub-second motion modifiers like the WHERE-shake and the teen-number wrist twist.
- Three-step retry: Flash@0s, Flash@0.5s, Flash-Lite@1.0s - covers transient 500s from Gemini without doubling the latency on the first try.
- Hard 30s asyncio.wait_for - guarantees the worker never holds the queue slot indefinitely. On timeout we return TIMEOUT to Redis and the user sees 'retry'.

9. Speech & Voice Design: ElevenLabs

services/speech.py is the smallest service in the backend, and intentionally so: ElevenLabs is the bottleneck for both audio in and audio out, and our job is to feed it cleanly and not block on it. The class wraps both directions because they share the same client and connection pool.

Speech-to-text: Scribe v1

The frontend records audio with expo-av's HIGH_QUALITY preset (m4a, AAC, 44.1kHz). The API forwards the file as a BytesIO into ElevenLabs' speech_to_text.convert with model_id=scribe_v1, language_code=en, tag_audio_events=False. We then strip parenthetical noise tags like '(audience laughing)' that occasionally slip through and collapse runs of whitespace.

```
_NOISE_RE = re.compile(r'[\(\[\<][^\)\]\>]{1,60}[\)\]\>]')
raw = result.text
text = re.sub(r'\s+', ' ', _NOISE_RE.sub('', raw)).strip()
```

Text-to-speech: Multilingual v2

TTS uses model_id=eleven_multilingual_v2 and voice Rachel (the widely-available default voice id 21m00Tcm4TlvDq8ikWAM, configurable via ELEVENLABS_VOICE_ID). The convert() call streams chunks; we join them in a thread (asyncio.to_thread) and write the resulting mp3 bytes to SeaweedFS at /audio/{video_id}.mp3.

Audio playback on the client

Once the polling loop sees status='done' with a truthy audio_url, the React Native client builds the URL via getAudioUrl(videoId), creates an Audio.Sound, calls playAsync, and registers a callback that unloads the sound on didJustFinish. We also expose a speaker icon on the bubble so the user can replay any past message.

```
// TranslatePage.jsx (excerpt)
const playAudio = useCallback(async (videoId) => {
  const { sound } = await Audio.Sound.createAsync({ uri: getAudioUrl(videoId) });
  await sound.playAsync();
  sound.setOnPlaybackStatusUpdate(s => {
    if (s.didJustFinish) sound.unloadAsync();
  });
}, []);
```

Failure mode that does not block the user

If TTS or audio upload fails, the worker still writes the recognition result to Redis with audio_url: null. The user gets the gloss + English text immediately and the speaker icon is hidden. We treat audio as a non-critical enhancement.

Voice Design: custom AI voices

The Voice Design feature lets users create a reusable voice persona through a two-step AI pipeline. The user types a natural language description; Gemini 2.5 Flash generates structured voice parameters (gender, age, accent, tone, pace); ElevenLabs Voice Design API instantiates the voice and returns a preview clip. The backend exposes two endpoints:

- POST /api/v1/voice/design - accepts { description } and returns { voice_id, name, labels, preview_url }. The Gemini call converts free-form text into ElevenLabs VoiceDesign parameters before the ElevenLabs call.
- POST /api/v1/voice/speak - accepts { voice_id, text } and streams mp3 audio synthesised with the custom voice. Used for the 'Read aloud' action on any message bubble when a custom voice is selected.

On the frontend, created voices are stored in SessionContext.createdVoices and surfaced in the VoicePickerModal under a 'My Voices' section alongside system OS voices grouped by language. Selecting a voice applies it to all 'Read aloud' actions across all tabs for the current session.

10. Frontend: Expo React Native App

The Expo client now has three top-level pages with shared state managed through SessionContext. Two custom hooks (useVideoUpload + useSpeechUpload) wrap the upload state machines, and a typed API client lives in lib/api.ts. SessionContext holds per-tab conversation history, the selected voice, and the created-voices list - the only cross-tab state that needs to survive tab switches.

Module layout

```
frontend/
  app/
    _layout.tsx            # root navigation shell
    index.tsx              # three-tab switcher (Translate/Animation/Voice)
    TranslatePage.jsx      # chat UI, sign + speech, history drawer
    AnimationPage.jsx      # sign.mt WebView + chat input + voice picker
    VoicePage.jsx          # voice design chat (describe -> create -> preview)
  components/
    camera-recorder.tsx    # CameraView + record + preflight detectHands
    video-preview.tsx      # VideoView + upload result panel
    history-drawer.tsx     # slide-in left panel (300px): conversation list
    voice-picker-modal.tsx # system voices by language + My Voices section
  contexts/
    SessionContext.tsx     # per-tab history, selectedVoice, createdVoices
  hooks/
    use-video-upload.ts    # POST /sign/recognize + poll /result
    use-speech-upload.ts   # record + POST /speech/transcribe + /gloss
  lib/
    api.ts                 # typed request() + endpoint helpers
```

SessionContext

SessionContext is a React Context that wraps the entire app (provided in _layout.tsx). It tracks three independent conversation histories - one per tab - so switching tabs never loses messages. It also holds the currently selected voice (system or custom) and the list of voices the user has created on the Voice tab. Conversation titles are auto-derived from the first message locally and confirmed via the backend title endpoint after the third message.

Camera recorder UX

The CameraRecorder component is responsible for everything between the user opening the camera and the worker receiving the file. It shows a live countdown badge (MAX_RECORD_SECONDS=10), a flash toggle (back camera only), a flip button, and a stop button. After recording it transitions to a preview view with a 'hands detected' badge whose colour reflects the pre-flight result.

- Front camera default - it is the natural framing for self-recording while talking to someone in front of you.
- Camera-ready promise: handleRecord awaits a 'camera ready' resolver before calling recordAsync to prevent the rare 'camera not initialised' iOS crash.
- MAX_RECORD_SECONDS=10 ceiling - keeps payloads small, pushes the user toward isolated, demoable signs, and bounds the inference cost.
- Hands badge colours: green (detected), amber (missing - retake?), red (check failed). The user can still proceed if they choose.

History drawer & voice picker

history-drawer.tsx slides in from the left (width 300px) and shows the current tab's conversation list with relative timestamps and a 'New' button. A voice-picker shortcut at the bottom opens the VoicePickerModal. voice-picker-modal.tsx groups system OS voices (via expo-speech) by language and shows a 'My Voices' section at the top for any AI-generated voices from the Voice tab. A preview button speaks a sample sentence in the selected voice before committing.

Speech upload hook & animation chrome injection

useSpeechUpload returns { isRecording, isProcessing, start, stop }. start() requests permissions, calls Audio.setAudioModeAsync, and creates a HIGH_QUALITY recording. stop() unloads the recording, posts the m4a to /speech/transcribe, then posts the transcript to /speech/gloss to enrich the message bubble with both the spoken sentence AND its ASL gloss back-translation. The AnimationPage's WebView injects two scripts (before and after Angular hydration) that insert a hide-chrome style tag and a MutationObserver so sign.mt's own UI never becomes visible.

11. Data, Storage and Conversation State

Postgres schema (db/models.py)

Two tables - one Conversation per session, many Messages per conversation. The Message row carries direction, content, gloss, audio_url and confidence. user_id is nullable so the schema is auth-ready without requiring auth on day 1.

```
class Conversation(Base):
    id: UUID PK = uuid4()
    session_id: UUID UNIQUE NOT NULL
    user_id: UUID NULL                      # ready for auth
    created_at: TIMESTAMPTZ NOT NULL

class Message(Base):
    id: UUID PK = uuid4()
    conversation_id: UUID FK -> conversations.id ON DELETE CASCADE
    direction: VARCHAR(32)   # 'deaf_to_hearing' | 'hearing_to_deaf'
    content: TEXT             # natural language string
    gloss: TEXT NULL          # ASL gloss
    audio_url: TEXT NULL      # SeaweedFS mp3 URL
    confidence: FLOAT NULL    # only set for sign recognition
    created_at: TIMESTAMPTZ NOT NULL
```

Session ID flow

The frontend generates a UUID once at module load (lib/api.ts SESSION_ID) and sends it on every request as the X-Session-ID header. The backend's upsert_conversation treats session_id as a unique key: first request creates the Conversation row, subsequent requests within the same session attach Messages to it. This is what makes the chat history endpoint return a sensible thread without authentication.

Object storage: SeaweedFS

We landed on SeaweedFS instead of MinIO/S3 because it ships as three tiny Go binaries (master, volume, filer), needs almost no config, and gives us HTTP-native upload and fetch with directory-style paths. services/storage.py exposes a single save_bytes helper with explicit folder + ext arguments so the same code path serves both video and audio:

```
# Sign video
await save_bytes(contents, file_id, 'video/mp4')
# -> /videos/{file_id}.mp4 on the filer

# TTS audio
await save_bytes(audio_bytes, video_id, 'audio/mpeg', folder='audio', ext='mp3')
# -> /audio/{video_id}.mp3 on the filer
```

Silent data collection (collector.py)

Every successful inference appends a record to data/raw/inferences.jsonl with the video_id, gloss, english, confidence and a UTC timestamp. Every human correction (POST /sign/corrections) appends to data/raw/corrections.jsonl. The intent is to use this corpus post-hackathon to fine-tune a Qwen3-VL 7B LoRA on real, in-the-wild samples. The I/O is done with asyncio.to_thread so a slow disk never blocks the loop.

12. Observability and Reliability

Even at 36 hours we wired up the full observability stack because demo failures are almost always 'something timed out and we don't know what'. The compose file ships Prometheus, Loki + Promtail, Grafana, and Langfuse for LLM tracing.

- Prometheus: prometheus-fastapi-instrumentator exposes /metrics with default HTTP histograms. We can answer 'p95 latency of /sign/recognize' in real time.
- Loki + Promtail: docker logs stream into Loki via promtail mounting /var/run/docker.sock - Grafana shows them with structured fields.
- Langfuse: each Gemini call is wrapped in a Langfuse trace so we can see prompt + response + latency per sign, indexed by gloss for spot-checking.
- python-json-logger: every log line is structured JSON, which means we can grep with jq and aggregate in Loki without log-format gymnastics.
- Health endpoint: /api/v1/health verifies API key presence so the demo can fail loudly at startup instead of mysteriously at first inference.

13. Latency Engineering: Hitting the 2-Second Bar

Two seconds end-to-end is the target. Below is the budget we measured - and the specific decisions taken to fit inside it.

Network upload (4MB H.264)	~150ms
API enqueues + 202 returned	~50ms
ffmpeg 480p transcode (worker)	~250ms
MediaPipe HandLandmarker VIDEO mode	~400ms
Gemini 2.5 Flash inline call	~900ms
ElevenLabs TTS (parallel-able, but serial today)	~400ms
Redis write + client poll round-trip	~80ms
Audio fetch + Audio.Sound.createAsync	~200ms
Total walltime visible to user	~1.6 - 2.0 seconds

Optimisations that actually moved the needle

- Shared tmp volume between API and worker to skip the SeaweedFS round-trip.
- ffmpeg ultrafast preset + crf 28 - we trade visual quality for upload speed.
- Inline base64 video in Gemini call (skips Files API ACTIVE polling).
- thinking_budget=0 - the single biggest Gemini latency win.
- MediaPipe face landmarks every 4th frame instead of every frame.
- ARQ keep_result=3600 + setex(3600) - results stay queryable for an hour even after the worker forgets them.
- Auto-play TTS on first poll where audio_url is truthy - removes a user tap.

14. Roadmap and Post-Hackathon Plans

We deliberately scoped the hackathon build to be a credible demo, not a finished product. The roadmap below is what turns ASL Bridge into something you'd actually carry in your pocket.

Phase 2 - Real signing avatar (4-8 weeks)

Replace the embedded sign.mt iframe with a self-hosted avatar generation pipeline based on Wan 2.6 via FAL.AI. This unlocks expressive faces, signer customisation, and lets us cache common phrases as static MP4s for instant playback.

Phase 3 - Multi-language sign support (2-3 months)

Integrate SignLLM and Prompt2Sign so the system handles British SL, Chinese SL and Korean SL alongside ASL. The current Gemini prompt structure is generic enough that we can swap the closed-set vocabulary per locale and reuse the rest of the pipeline.

Phase 4 - Fine-tuned ASL recognition

Once enough corrections accumulate in data/raw/corrections.jsonl, we will fine-tune Qwen3-VL 7B with LoRA on (video, landmark JSON, gloss) triples. This is the path off the Gemini API for both cost and offline-capability reasons. The data pipeline is already collecting samples in the right format from day one.

Phase 5 - User accounts + offline mode

user_id on conversations is already nullable - a JWT middleware + users table is roughly half a day. After auth we can add shareable conversation links, multi-device sync, and an offline mode running the recogniser on-device via ONNX once the fine-tuned model fits in mobile memory.

Open questions we did not have time to answer

- Can we run MediaPipe on-device and avoid uploading the video? Probably, but the prompt currently relies on landmarks AND the full clip.
- Right chunking strategy for continuous, multi-sentence ASL? Today we expect single-utterance clips.
- Adversarial robustness: lighting, dark skin tones, partial occlusion, off-axis framing - none stress-tested yet.

15. Team, Credits and Closing Notes

ASL Bridge was built over 36 hours at ConHacks 2026 by a four-person team. Each engineer owned one slice of the system end-to-end, with a daily integration sync to keep the moving parts compatible.

Team

Kai - Frontend	Expo app, camera + mic flows, ElevenLabs TTS playback, three-tab navigation, session ID, animation WebView chrome injection, Voice Design page, SessionContext, history drawer, voice picker modal.
Bob - Computer Vision	OpenCV preprocessing, MediaPipe landmark sequences, face anchor design, velocity-based motion features.
Max - UI / UX	Design system, three-mode layout, confidence indicator, voice reasoning chips, demo polish, art direction, brand and visual identity.
Dave - Backend + Models	FastAPI + ARQ, Gemini prompt engineering, ElevenLabs STT + Voice Design API, conversation persistence, voice router, observability stack, data collection.

Acknowledgements

- Google MediaPipe team for the HandLandmarker and FaceLandmarker tasks - battery-efficient, ridiculously easy to integrate, runs everywhere.
- Google DeepMind for Gemini 2.5 Flash - the only frontier vision model with low enough latency for a real-time conversational demo.
- ElevenLabs for both Scribe (STT) and the Multilingual v2 voice that makes the demo feel human, not robotic.
- sign.mt for the open-source ASL avatar viewer that powers Phase 1 of the hearing-to-deaf path.
- The Expo team for making it possible for four engineers to ship a cross-platform mobile app inside a 36-hour window.

Repository

Source: github.com/naik26m3/signly-conhacks-2026
Backend Swagger: http://localhost:8000/docs
Backend OpenAPI spec: http://localhost:8000/openapi.json
Run locally: `docker compose up --build` (after copying .env.example to .env and filling in GEMINI_API_KEY + ELEVENLABS_API_KEY).

Closing thought

The hard part was never any single component - MediaPipe is great, Gemini is great, ElevenLabs is great. The hard part was making them all finish in two seconds, persist properly, and feel like a conversation rather than a series of API calls. We think we got there. We hope you agree.